

Freie Universität Berlin

Bachelorarbeit am Institut für Informatik der Freien Universität Berlin

Arbeitsgruppe Cybersecurity and AI

Reduzierung von False Positives in der Cloud-Sicherheit durch hybride Analyse von Metriken und Log-Kontext: Eine skalierbare Architektur-Studie auf CloudAnoBench

Nirosh Heintze

Matrikelnummer: 5585800

niroh04@zedat.fu-berlin.de

Betreuer/in: Hamed H. Fard

Eingereicht bei: Prof. Dr.-Ing. habil. Gerhard Wunder

Berlin, 15. Juni 2026

Zusammenfassung

Automatisierte Anomalie-Erkennung in Cloud-Infrastrukturen erzeugt häufig eine hohe Rate an Fehlalarmen (*False Positives*), die Analysten belasten und die Reaktionsfähigkeit auf echte Sicherheitsvorfälle beeinträchtigen. Diese Arbeit untersucht auf dem CloudAnoBench-Datensatz, inwiefern Log-Kontext die False-Positive-Rate einer metrics-basierten Detection reduzieren kann. Dazu wird eine reproduzierbare Case-Level-Pipeline entwickelt, die einen XGBoost-Klassifikator auf aggregierten Systemmetriken, eine log-basierte TF-IDF-Repräsentation mit logistischer Regression sowie zwei einfache Fusionsvarianten vergleicht. Die finale Evaluation erfolgt leakage-sicher mit einem scenario-aware Train-Validation-Test-Split; Early Stopping und Threshold-Wahl werden ausschließlich auf dem Validierungsset vorgenommen, während das Testset nur der finalen Auswertung dient.

Die Ergebnisse zeigen, dass die metrics-basierte Baseline auf Case-Ebene zwar alle Angriffs-Cases erkennt ($\text{Recall}_{\text{mali}} = 1,0000$), aber zugleich alle negativen Test-Cases als positiv klassifiziert ($\text{FPR}_{\text{total}} = 1,0000$). Die log-basierte Komponente liefert die beste Trennleistung und reduziert die False-Positive-Rate auf 0,2537 bei einem Test-Recall von 0,9375; getrennt nach Negativtypen ergeben sich $\text{FPR}_{\text{anom}} = 0,2368$ und $\text{FPR}_{\text{norm}} = 0,2636$. Die AND-Fusion ist funktional identisch zur Log-only-Variante, da der metrische Trigger bei allen 253 Test-Cases aktiv ist; die Soft-Fusion verschlechtert sich gegenüber Log-only leicht. Damit trägt auf diesem Benchmark primär der Log-Kontext zur False-Positive-Reduktion bei, während die untersuchten Metrik-Aggregate und Fusionsregeln keinen zusätzlichen Nutzen liefern. Die angestrebte Recall-Schwelle von 0,95 wird im finalen Testsplitt knapp verfehlt und wird als zentrale Limitation diskutiert.

Eidesstattliche Erklärung

Ich versichere hiermit an Eides Statt, dass diese Arbeit von niemand anderem als meiner Person verfasst worden ist. Alle verwendeten Hilfsmittel wie Berichte, Bücher, Internetseiten oder ähnliches sind im Literaturverzeichnis angegeben, Zitate aus fremden Arbeiten sind als solche kenntlich gemacht. Die Arbeit wurde bisher in gleicher oder ähnlicher Form keiner anderen Prüfungskommission vorgelegt und auch nicht veröffentlicht.

15. Juni 2026

Nirosh Heintze

Inhaltsverzeichnis

1	Einleitung	7
1.1	Motivation	7
1.2	Problemstellung	7
1.3	Zielsetzung und Forschungsfrage	7
1.4	Beiträge der Arbeit	8
1.5	Aufbau der Arbeit	9
2	Grundlagen und verwandte Arbeiten	9
2.1	Cloud-Anomalieerkennung und Alert Fatigue	9
2.2	Metrik- und log-basierte Anomalieerkennung	10
2.3	Unbalancierte Klassifikation und Precision-Recall-Metriken	10
2.4	XGBoost für tabellarische Metrikdaten	11
2.5	TF-IDF und logistische Regression für Logdaten	11
2.6	Data Leakage und gruppenbasierte Validierung	11
2.7	Klassifikatorfusion und Case-Level-Aggregation	12
3	Datensatz und Datenaufbereitung	13
3.1	CloudAnoBench und Datenbasis	13
3.2	Datenqualität und Preprocessing	13
3.3	Canonical Feature Set und universelle Features	14
3.4	Leakage-Vermeidung und Split-Strategie	14
4	Metrik-basierte Baseline	15
4.1	Modellwahl und Hyperparameter	15
4.2	Schema-Leakage, Feature-Reduktion und Case-Level-Aggregation . . .	16
4.3	Ergebnisse der Metrik-Baseline	16
5	Log-basierte Verifikation	17
5.1	Log-Daten auf Case-Ebene	17
5.2	Log-Normalisierung	18
5.3	TF-IDF-Vektorisierung	18
5.4	Logistische Regression	19
5.5	Ergebnisse der Log-Komponente	20
5.6	Threshold-Sensitivity-Analyse	20
6	Hybride Fusion und Evaluation	21
6.1	Fusion-Eingaben auf Case-Ebene	21
6.2	AND-Gate-Fusion und Soft Score	22
6.3	Ablation-Studie	23
6.4	AND-Gate-Degeneration	24
6.5	Qualitätssicherung und Reproduzierbarkeit	24

7	Diskussion und Limitationen	26
7.1	Beantwortung der Forschungsfrage	26
7.2	Rolle des Log-Kontexts	26
7.3	Grenzen der Metrik-Komponente	26
7.4	Methodische Limitationen	27
7.5	Skalierbarkeit als konzeptuelle Einordnung	28
8	Fazit und Ausblick	28
8.1	Zusammenfassung	28
8.2	Beantwortung der Forschungsfrage	29
8.3	Ausblick	29
A	Anhang	32

1 Einleitung

1.1 Motivation

Automatisierte Anomalie-Erkennung ist ein zentrales Werkzeug im Betrieb großer Cloud-Infrastrukturen. Monitoring-Systeme werten kontinuierlich Systemmetriken aus und lösen bei auffälligen Abweichungen Alerts aus. In der Praxis führt dieser Ansatz jedoch zu einem hohen Aufkommen an Fehlalarmen (*False Positives*): Betriebsschwankungen, reguläre Lastspitzen oder geplante Batch-Jobs erzeugen Metrikanomalienmuster, die denen echter Angriffe oder Systemfehler ähneln, ohne es zu sein.

Dieses Phänomen ist in der Literatur als *Alert Fatigue* bekannt und stellt ein anerkanntes operatives Problem im Sicherheitsbetrieb dar. Es beeinträchtigt die Reaktionsfähigkeit von Analysten auf tatsächliche Vorfälle. Die Reduktion der False-Positive-Rate — ohne dabei die Erkennungsrate sicherheitsrelevanter Ereignisse wesentlich zu senken — ist daher eine zentrale Anforderung an praxistaugliche Erkennungssysteme. Abschnitt 2.1 ordnet diesen Befund in die Literatur ein.

1.2 Problemstellung

Ein grundlegendes Problem metrischer Anomalie-Detektoren liegt im Zielkonflikt zwischen Recall und False-Positive-Rate. Um sicherzustellen, dass keine echten Angriffe unentdeckt bleiben (hoher Recall), muss der Entscheidungs-Threshold des Detektors niedrig gesetzt werden. Ein niedriger Threshold markiert jedoch zwangsläufig auch viele tatsächlich unbedenkliche Instanzen als auffällig, was die FPR in die Höhe treibt. Dieser Trade-off ist bei der Verwendung von wenigen, universell verfügbaren Metriken besonders ausgeprägt, da solche Features allein häufig keine ausreichende Trennschärfe zwischen malignen und benignen Fällen bieten.

Log-Dateien liefern einen ergänzenden, qualitativ anderen Informationstyp: Sie enthalten prozessuale Beschreibungen von Systemereignissen, die den Kontext einer Metrik-Auffälligkeit klären können [7]. Eine Lastspitze in der CPU-Auslastung kann sowohl bei einem malignen Prozess als auch bei einem legitimen Backup-Job auftreten — der zugehörige Log-Eintrag hingegen unterscheidet beide Fälle in der Regel klar. Log-Daten eignen sich daher als nachgelagerte Verifikationsstufe: Alerts, die metrisch ausgelöst wurden, können auf Case-Ebene anhand ihres Log-Kontexts gefiltert werden.

Die Kombination beider Signalquellen zu einer hybriden Pipeline — einem metrischen Trigger gefolgt von einer log-basierten Verifikation — ist konzeptuell naheliegend. CloudAnoBench [17] stellt einen synthetischen Benchmark bereit, der für jeden überwachten Case sowohl Metriken als auch Log-Dateien bereitstellt und damit eine kontrollierte Evaluation eines solchen zweistufigen Ansatzes ermöglicht.

1.3 Zielsetzung und Forschungsfrage

Ziel dieser Arbeit ist die Entwicklung und Evaluation einer zweistufigen hybriden Anomalie-Erkennungspipeline auf CloudAnoBench. Der Schwerpunkt liegt auf einer methodisch kontrollierten Evaluation und der expliziten Diskussion der Grenzen des Ansatzes — einschließlich unerwarteter Befunde.

1. Einleitung

Die zentrale Forschungsfrage lautet:

Kann eine zweistufige hybride Pipeline — bestehend aus einem metrischen Anomalie-Detektor und einer log-basierten Nachfilterung — die False-Positive-Rate auf dem CloudAnoBench-Datensatz deutlich senken, ohne die Recall-Rate unter 0,95 zu drücken?

Die Arbeit untersucht diese Frage durch einen kontrollierten Ablations-Aufbau, der eine rein metrische Baseline, eine log-basierte Komponente sowie zwei Fusionsstrategien — ein binäres AND-Gate und eine Soft-Fusion — miteinander vergleicht. Die Evaluation folgt dabei strikten Anforderungen an die Vermeidung von Datenleck und eine leakage-sichere Trainings-Test-Trennung.

Der Begriff *Architektur-Studie* im Titel bezieht sich auf die konzeptuelle Zerlegung der Pipeline in metrischen Trigger, log-basierte Verifikation und Fusionslogik; eine verteilte Implementierung oder messtechnische Skalierungsanalyse ist nicht Gegenstand dieser Arbeit (vgl. Abschnitt 7.5).

1.4 Beiträge der Arbeit

Die vorliegende Arbeit leistet die folgenden konkreten Beiträge:

1. **Reproduzierbare Case-Level-Pipeline auf CloudAnoBench.** Es wird eine in Notebooks dokumentierte, reproduzierbare Pipeline entwickelt, die alle Phasen — Datenaufbereitung, Case-Level-Feature-Engineering, Modelltraining, Fusion und Evaluation — auf CloudAnoBench umsetzt. Alle Ergebnisse sind durch 43 Unit-Tests und `random_state=42` reproduzierbar und nachvollziehbar.
2. **Case-Level Metrik-Baseline mit XGBoost auf aggregierten Features.** Es wird eine metrische Baseline mit XGBoost [4] entwickelt, die direkt auf 25 case-aggregierten Features der fünf universellen Systemmetriken trainiert wird (Mittelwert, Maximum, Standardabweichung, 95. Perzentil und Trend je Metrik). Die Beschränkung auf die fünf universell verfügbaren Features ist eine methodisch begründete Entscheidung zur Vermeidung von Target Leakage durch strukturelle Fehlwerte.
3. **Log-basierte Verifikation mit Normalisierung, TF-IDF und logistischer Regression.** Es wird eine Case-Ebene-Komponente implementiert, die Log-Texte vor der TF-IDF-Vektorisierung durch eine regelbasierte Normalisierung synthetischer Artefakte bereinigt (Zeitstempel, IP-Adressen, PIDs, Ports, isolierte Zahlen werden durch generische Token ersetzt). Die logistische Regression auf dem normalisierten TF-IDF-Raum wird als interpretierbare Baseline eingesetzt; das Vokabular wird ausschließlich auf Trainingsdaten gefittet.
4. **Ablation der Fusionsstrategien einschließlich Degenerationsbefund.** Es werden zwei Fusionsstrategien — AND-Gate und Soft-Fusion — evaluiert und mit den Einzelkomponenten verglichen. Ein zentraler empirischer Befund ist die vollständige Degeneration des AND-Gates: Der metrische Detektor löst beim gewählten Threshold alle 253 Test-Cases aus (100 %), sodass das AND-Gate

funktional äquivalent zur reinen Log-Filterung wird. Soft-Fusion verschlechtert sich gegenüber Log-only. Dieser Befund wird in Abschnitt 6.4 explizit diskutiert.

1.5 Aufbau der Arbeit

Abschnitt 2 legt die konzeptionellen und methodischen Grundlagen der Pipeline dar. Behandelt werden Cloud-Anomalieerkennung, unbalancierte Klassifikation und Precision-Recall-Metriken, XGBoost, TF-IDF, Data Leakage und gruppenbasierte Validierung sowie Klassifikatorfusion und Case-Level-Aggregation.

Abschnitt 3 beschreibt die Pipeline in vier Phasen: Datenaufbereitung und Leakage-Vermeidung, metrische Baseline, log-basierte Verifikation sowie hybride Fusion und Ablation. Die Ergebnisse werden durch Tabellen und Abbildungen dokumentiert; unerwartete Befunde — insbesondere die AND-Gate-Degeneration — werden als eigenständige Abschnitte behandelt.

Abschnitt 4 diskutiert die Ergebnisse im Hinblick auf die Forschungsfrage, bewertet die Rolle der einzelnen Komponenten, benennt die methodischen Limitationen der Arbeit und gibt einen Ausblick auf mögliche Erweiterungen.

2 Grundlagen und verwandte Arbeiten

2.1 Cloud-Anomalieerkennung und Alert Fatigue

Moderne Cloud-Umgebungen werden kontinuierlich über systemseitige Metriken wie CPU-Auslastung, Speicherverbrauch und Netzwerkdurchsatz überwacht. Anomalie-Detektoren werten diese Zeitreihendaten automatisiert aus und generieren Alerts bei auffälligen Abweichungen. In großen, dynamischen Infrastrukturen erzeugen solche Systeme jedoch regelmäßig eine hohe Zahl von Fehlalarmen — sogenannte False Positives — die Sicherheitsanalysten belasten und deren Reaktionsfähigkeit auf echte Vorfälle verringern. Dieser Effekt ist in der Literatur als *Alert Fatigue* bekannt und stellt ein anerkanntes operatives Problem in Security Operations Centres dar [15]. Alahmadi et al. [1] belegen in einer qualitativen Studie, dass in real betriebenen SOC's ein erheblicher Teil der ausgelösten Sicherheitsalarme auf False Positives entfällt, was die Analysekapazität für tatsächliche Vorfälle systematisch einschränkt.

Die False-Positive-Rate (FPR) ist dabei die zentrale operative Zielgröße: Sie beschreibt den Anteil tatsächlich negativer Instanzen, der fälschlicherweise als Alarm eingestuft wird. Eine hohe FPR verursacht nicht nur unnötigen Analyseaufwand, sondern kann im Extremfall dazu führen, dass echte Angriffe im Rauschen untergehen. Die Reduktion der FPR bei gleichzeitiger Aufrechterhaltung einer hohen Erkennungsrate (Recall) bildet daher die Forschungsfrage dieser Arbeit.

Als Evaluierungsgrundlage dient CloudAnoBench [17], ein synthetischer Benchmark, der Systemmetriken und Log-Dateien für Szenarien mit malignem, anomalem und normalem Systemverhalten bereitstellt. CloudAnoBench ist ein neuerer Benchmark und nicht als langjährig etablierter Goldstandard der Community zu verstehen; die in dieser Arbeit erzielten Ergebnisse sind datensatzspezifisch und besitzen keine unmittelbare Allgemeingültigkeit.

2.2 Metrik- und log-basierte Anomalieerkennung

Anomalieerkennung in Cloud-Systemen kann grundsätzlich auf zwei komplementären Informationsquellen operieren: Systemmetriken und Log-Dateien.

Systemmetriken sind numerische Zeitreihendaten, die den Ressourcenzustand eines Systems zu diskreten Zeitpunkten quantifizieren — etwa Prozessorauslastung, Speicherverbrauch oder Netzwerkdurchsatz. Sie sind gut strukturiert, hochfrequent verfügbar und lassen sich effizient mit tabellarischen Lernverfahren verarbeiten. Ihre Schwäche besteht darin, dass unterschiedliche Ereignistypen — etwa legitime Lastspitzen und Angriffsmuster — identische Metrikverläufe erzeugen können. Metriken allein erlauben in solchen Fällen keine zuverlässige Unterscheidung.

Log-Dateien hingegen enthalten textuelle Beschreibungen von Systemereignissen, Prozessaufrufen und Fehlerzuständen. Sie liefern einen prozessualen Kontext, der über die bloße Ressourcensicht hinausgeht. He et al. [7] zeigen in einer umfassenden Übersicht, dass automatisierte Log-Analyse ein etabliertes Werkzeug im Reliability Engineering ist: Textmuster in Log-Dateien können charakteristische Signaturen für unterschiedliche Ereigniskategorien tragen, die in den Metriken nicht sichtbar sind.

Die vorliegende Arbeit verfolgt einen zweistufigen Ansatz: Ein metrischer Detektor identifiziert zunächst potenzielle Auffälligkeiten auf Basis von Ressourcensignalen; eine log-basierte Komponente verifiziert anschließend auf Case-Ebene, ob der Log-Kontext den Alert stützt. Beide Komponenten sind methodisch aufeinander abgestimmt und werden in Abschnitt 3 im Detail beschrieben.

2.3 Unbalancierte Klassifikation und Precision-Recall-Metriken

Anomalie-Erkennungsprobleme sind typischerweise stark unbalanciert: Positive Instanzen (Angriffe, Fehler) sind weit seltener als negative (Normalbetrieb). He und Garcia [6] zeigen, dass in solchen Szenarien standardmäßige Klassifikationsmetriken wie Accuracy irreführend sind, da sie von der Mehrheitsklasse dominiert werden.

Die für diese Arbeit relevanten Metriken sind:

Precision $P = \frac{TP}{TP+FP}$: Anteil der korrekt als positiv klassifizierten Instanzen an allen als positiv gekennzeichneten.

Recall $R = \frac{TP}{TP+FN}$: Anteil der tatsächlich positiven Instanzen, die erkannt werden. In dieser Arbeit als harte Nebenbedingung ($R \geq 0,95$) formuliert.

False-Positive-Rate $FPR = \frac{FP}{FP+TN}$: Anteil der negativen Instanzen, die fälschlicherweise als positiv eingestuft werden. Dies ist die zentrale operative Zielgröße.

Average Precision (AP) Fläche unter der Precision-Recall-Kurve, berechnet als gewichteter Mittelwert der Precision-Werte über alle Recall-Stufen. AP fasst die Klassifikatorleistung über alle möglichen Thresholds zusammen.

Davis und Goadrich [5] zeigen, dass die PR-Kurve gegenüber der ROC-Kurve bei unbalancierten Problemen informativer ist: Während ROC-AUC die Leistung im negativen Regime mitunter überschätzt, macht die PR-Kurve Schwächen bei der Behandlung der Minderheitsklasse direkt sichtbar. Saito und Rehmsmeier [14] bestätigen

dies und empfehlen PR-Kurven explizit für stark unbalancierte Klassifikationsprobleme. Diese Befunde motivieren die Verwendung von AUCPR als primäre Optimierungsmetrik in Phase 2 sowie die Fokussierung auf FPR und AP in der Evaluation.

2.4 XGBoost für tabellarische Metrikdaten

Als metrischer Basis-Detektor wird in dieser Arbeit XGBoost [4] eingesetzt. XGBoost implementiert Gradient Tree Boosting und approximiert den Split-Finding-Algorithmus durch eine histogrammbasierte Methode (`tree_method='hist'`), die den Trainingsaufwand gegenüber dem exakten Verfahren deutlich reduziert. Für tabellarisch strukturierte Daten mit numerischen Features und heterogenen Skalen stellt XGBoost ein in der Literatur breit erprobtes Verfahren dar.

Relevante Eigenschaften für diese Arbeit sind der Parameter `scale_pos_weight` zur Gewichtung der Minderheitsklasse im unbalancierten Setting sowie die Möglichkeit, AUCPR direkt als Optimierungsmetrik zu verwenden.

XGBoost wird hier nicht als grundsätzlich überlegenes Verfahren positioniert, sondern als solide, reproduzierbare Baseline für tabellarische Klassifikation. Es ist nicht Ziel dieser Arbeit, das metrische Modell zu optimieren; vielmehr soll seine Leistungsgrenze die Notwendigkeit einer ergänzenden Log-Komponente motivieren.

2.5 TF-IDF und logistische Regression für Logdaten

Die log-basierte Komponente dieser Arbeit nutzt eine Kombination aus *TF-IDF*-Vektorisierung und logistischer Regression. *TF-IDF* (*Term Frequency – Inverse Document Frequency*) gewichtet Terme proportional zu ihrer Häufigkeit im Dokument, normiert jedoch durch ihre Häufigkeit über alle Dokumente. Häufige, wenig diskriminierende Terme — in Log-Dateien etwa routinemäßige Statusmeldungen — erhalten dadurch geringere Gewichte; seltene, charakteristische Terme werden hervorgehoben.

Auf der TF-IDF-Matrix wird eine logistische Regression trainiert. Lineare Modelle auf sparse hochdimensionalen Textrepräsentationen sind für diese Problemklasse eine etablierte Wahl; umgesetzt wird der Klassifikator in dieser Arbeit mit scikit-learn [12]. Die logistische Regression liefert Wahrscheinlichkeitsschätzungen, die in der nachgelagerten Fusionsstufe direkt weiterverwendet werden.

Die Entscheidung für TF-IDF und logistische Regression folgt dem Prinzip einer interpretierbaren und effizient trainierbaren Baseline. Fortgeschrittenere Ansätze zur Log-Analyse — etwa struktur-bewusste Log-Parser, vortrainierte Sprachmodelle oder Embedding-Methoden — wurden in dieser Arbeit nicht untersucht. Sie stellen eine naheliegende Erweiterung dar, sind aber nicht Gegenstand der vorliegenden Evaluation.

2.6 Data Leakage und gruppenbasierte Validierung

Eine korrekte Evaluation maschineller Lernverfahren setzt voraus, dass keine Information aus dem Testset in den Trainingsprozess einfließt. Kaufman et al. [9] klassifizieren verschiedene Formen von Datenleck (*Data Leakage*) und zeigen, dass selbst scheinbar harmlose Preprocessing-Schritte — etwa eine globale Skalierung oder Im-

putation — die gemessene Testleistung künstlich erhöhen können, wenn sie auf dem Gesamtdatensatz statt ausschließlich auf dem Trainingsanteil durchgeführt werden.

Für diese Arbeit ist eine spezifische Form von Leakage besonders relevant: *Schema-Leakage* durch strukturelle Fehlwerte. Werden verschiedene Klassen mit systematisch unterschiedlichen Feature-Schemata erfasst, entstehen typspezifische NaN-Muster, die ein Modell implizit zur Bestimmung der Klassenzugehörigkeit nutzen kann — noch bevor inhaltlich relevante Signale verarbeitet werden. Kaufman et al. [9] klassifizieren dies als Target Leakage, da die Feature-Struktur die Zielvariable enkodiert und eine valide Evaluation verhindert. Die methodische Konsequenz ist die Beschränkung der Modellierung auf Features, die in allen Klassen vollständig verfügbar sind.

Ein zweites Leakage-Risiko ergibt sich aus der Datensatzstruktur: Beobachtungen innerhalb desselben Szenarios sind strukturell korreliert. Eine zufällige Aufteilung in Trainings- und Testmenge ohne Berücksichtigung dieser Gruppen würde Szenarien auf beiden Seiten des Splits erlauben und die Trennleistung systematisch überschätzen. Roberts et al. [13] zeigen, dass bei Daten mit gruppenweise korrelierten Beobachtungen eine gruppenweise Partitionierung notwendig ist. In dieser Arbeit wird `GroupShuffleSplit` verwendet, das sicherstellt, dass kein Szenario gleichzeitig in Trainings- und Testmenge vertreten ist.

2.7 Klassifikatorfusion und Case-Level-Aggregation

Die Kombination mehrerer Klassifikatoren zu einem gemeinsamen Entscheidungssystem ist ein etabliertes Forschungsfeld. Kittler et al. [10] systematisieren verschiedene Fusionsregeln — darunter Produkt-Regel, Summen-Regel und Mehrheitsentscheid — und zeigen, unter welchen Voraussetzungen sie asymptotisch äquivalent sind. Das in dieser Arbeit verwendete AND-Gate stellt eine konservative Fusionsstrategie dar: Ein Alert wird nur dann ausgegeben, wenn *beide* Komponenten — metrischer Detektor und Log-Klassifikator — unabhängig voneinander positiv entscheiden. Das AND-Gate maximiert damit die Präzision auf Kosten des Recalls. Ob diese Strategie im konkreten Evaluationsszenario einen Mehrwert gegenüber einer Einzelkomponente liefert, ist eine empirische Frage, die in Abschnitt 6.3 untersucht wird.

Eine eigene methodische Herausforderung ergibt sich aus der unterschiedlichen Granularität beider Komponenten: Der metrische Detektor operiert auf Zeilenebene (einzelne Messpunkte), die Log-Analyse hingegen auf Case-Ebene (eine Log-Datei pro Case). Um beide Ebenen zu vereinen, müssen die zeilenweisen Scores des metrischen Detektors auf Case-Ebene aggregiert werden. In dieser Arbeit wird das Maximum der zeilenweisen Wahrscheinlichkeiten eines Cases verwendet:

$$\hat{p}_{\text{case}} = \max_{i \in \text{case}} \hat{p}_i$$

Diese Max-Aggregation entspricht der Annahme, dass ein einzelner anomaler Messpunkt innerhalb eines Cases als hinreichendes Indiz für eine Auffälligkeit gewertet wird. Konzeptuell ist dieses Prinzip verwandt mit der Instanz-Annahme im *Multiple-Instance Learning* [11]: Ein Bag (hier: Case) gilt als positiv, wenn mindestens eine seiner Instanzen (hier: Zeilen) positiv ist. Die Max-Aggregation ist keine optimierte Fusionsstrategie, sondern eine methodisch begründbare und einfach implementierbare Heuristik.

3 Datensatz und Datenaufbereitung

3.1 CloudAnoBench und Datenbasis

Grundlage der Untersuchung ist CloudAnoBench [17], ein synthetischer Benchmark zur multimodalen Anomalieerkennung in Cloud-Umgebungen. Der Datensatz stellt für jede Beobachtungseinheit sowohl Zeitreihendaten von Systemmetriken als auch zugehörige Log-Dateien bereit und eignet sich damit explizit für eine hybride Analyse, wie sie in dieser Arbeit untersucht wird. Es sei ausdrücklich darauf hingewiesen, dass CloudAnoBench ein neuerer Benchmark ist, der noch nicht als etablierter Goldstandard der Community gilt; die Ergebnisse dieser Arbeit sind daher im Kontext dieses spezifischen Datensatzes zu interpretieren.

Der Datensatz umfasst drei inhaltlich distinkte Typen: *mali* (maliciously anomalous), *anom* (non-malicious anomalous) und *norm* (normal). Die Unterscheidung ist für die binäre Klassifikationsaufgabe relevant, da nur *mali*-Instanzen als positiv (Label 1) und *anom* sowie *norm* als negativ (Label 0) kodiert werden. *Anom*-Fälle stellen dabei nicht-maliziöse technische Anomalien dar, die von einem Sicherheitsdetektor korrekt als unbedenklich eingestuft werden sollen; eine hohe Verwechselungsrate zwischen *mali* und *anom* wäre operativ besonders kostspielig [17]. *Norm*-Cases repräsentieren planmäßigen Betrieb, der aufgrund ressourcenintensiver, aber nicht-maliziöser Aktivitäten — etwa Backup- oder Batch-Prozesse — metrisch auffällig wirken kann. Solche Fälle wurden im Kontext dieser Arbeit als besonders relevante Quelle von Fehlalarmen identifiziert; in der Evaluation werden sie über eine separate FPR_{norm} ausgewertet.

Tabelle 1: Datenbasis: Zeilen und Cases nach Typ

Typ	Zeilen (roh)	Zeilen (clean)	Cases	Label
mali	19 385	19 384	223	positiv (1)
anom	41 400	41 400	460	negativ (0)
norm	51 208	51 208	569	negativ (0)
Gesamt	111 993	111 992	1 252	

Tabelle 1 fasst die Datenbasis zusammen. Lediglich eine einzige Zeile in *mali* wurde beim Bereinigungsschritt entfernt (0,01 %), was die Datenqualität als insgesamt hoch ausweist. Allen 1 252 Cases ist genau eine Log-Datei zugeordnet; kein Fall weist eine fehlende Log-Datei auf. Der Datensatz deckt insgesamt 45 verschiedene Szenarien ab [17].

3.2 Datenqualität und Preprocessing

Die Rohdaten liegen im CSV-Format vor und weisen zwei typische Qualitätsprobleme auf. Erstens enthalten einige Felder residuale Anführungszeichen aus einer überlagerten CSV-Kodierung (Parameter `quoting=3` beim Einlesen), die durch eine explizite Zeichenkettenbereinigung entfernt werden. Zweitens sind numerische Felder in einigen Dateien als Zeichenketten kodiert und müssen in Fließkommazahlen überführt werden.

3. Datensatz und Datenaufbereitung

Die Median-Imputation fehlender Werte erfolgt in zwei inhaltlich getrennten Kontexten: In Phase 1 wird für die explorative Datenanalyse und die Erstellung der bereinigten Parquet-Artefakte eine Self-Fit-Imputation auf den jeweils vollständig verfügbaren Typ-Daten durchgeführt. Dieser Schritt dient ausschließlich der Datenvorbereitung und Visualisierung; er hat keinen Evaluationsbezug. Für alle evaluationsrelevanten Schritte in Phase 2 und Phase 4 werden die Medianwerte hingegen ausschließlich auf dem Trainingsanteil berechnet, in `data/processed/mali_train_medians.json` gespeichert und danach ohne Neufitting auf die Testdaten angewendet. Diese Trennung schützt vor einem verbreiteten Fehler bei der Vorverarbeitung, der in der Literatur als *data leakage* bezeichnet wird [9].

Die Wahl des Medians gegenüber dem arithmetischen Mittel ist durch die Verteilung der Metriken motiviert: Systemmetriken wie `net_out` zeigen häufig eine rechtschiefe Verteilung mit einzelnen Ausreißerspitzen, gegen die der Median robust ist [8].

3.3 Canonical Feature Set und universelle Features

Jeder der drei Datensatztypen wird mit einem unterschiedlichen Satz von Systemmetriken erfasst, was auf unterschiedliche Monitoring-Konfigurationen je Datensatztyp hindeutet [17]. Um eine einheitliche Datengrundlage zu schaffen, wird eine *kanonische Feature-Matrix* mit 34 Features aufgebaut, die die Vereinigung aller je gemessenen Metriken darstellt. Fehlende Spalten werden mit NaN aufgefüllt.

Von diesen 34 Features sind lediglich fünf in allen drei Datensatztypen vorhanden:

`cpu_usage`, `mem_usage`, `disk_io`, `net_in`, `net_out`

Die verbleibenden 29 Features sind typspezifisch und strukturell in mindestens einem Typ vollständig als NaN belegt: In mali fehlen 28 von 34 Features strukturell, in anom 23 und in norm lediglich 6. Das Ausmaß dieser strukturellen Fehlwerte ist kein Zufallsartefakt, sondern spiegelt direkt die unterschiedlichen Erhebungsmethoden je Datensatztyp wider. Abbildung 1 zeigt die Verteilung der fünf universellen Features getrennt nach Klasse.

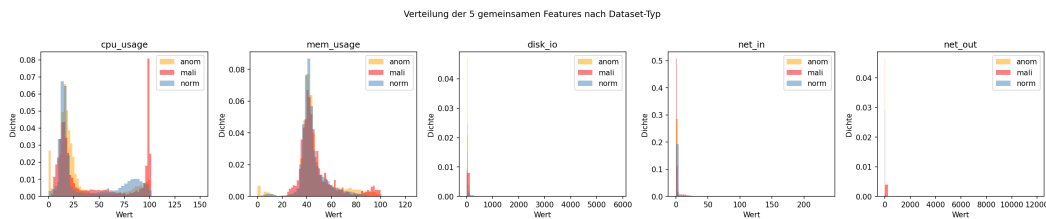


Abbildung 1: Verteilung der fünf universellen Features nach Klasse (positiv = mali, negativ = anom/norm).

3.4 Leakage-Vermeidung und Split-Strategie

Für eine leakage-sichere Evaluation werden die Daten in drei disjunkte Teilmengen aufgeteilt. Als Gruppenkennung dient die Kombination aus `dataset_type` und `scenario_id` (`group_id`); kein Szenario erscheint in mehr als einer Teilmenge. Die

Aufteilung erfolgt in zwei Stufen mit `GroupShuffleSplit` [13]: In einem ersten äußeren Schritt werden ca. 20 % der Gruppen als Testset abgetrennt; in einem zweiten inneren Schritt werden aus dem verbleibenden Pool ca. 20 % als Validierungsset reserviert. Das Trainingsset dient ausschließlich dem Modelltraining und der Imputation; das Validierungsset wird für Early Stopping und die Threshold-Wahl verwendet; das Testset wird ausschließlich zur finalen Evaluation eingesetzt. Die Nicht-Überlappung aller drei Splits nach Gruppenkennung wurde artefaktseitig verifiziert (`docs/split_assignments.csv`). Beobachtungen innerhalb desselben Szenarios sind strukturell korreliert; eine zufällige Aufteilung ohne Berücksichtigung dieser Gruppenstruktur würde die Trennleistung im Test systematisch überschätzen.

Tabelle 2 fasst die resultierenden Teilmengen zusammen.

Tabelle 2: Gruppenbasierter Train/Validation/Test-Split (`random_state=42`)

Split	Cases	Gruppen	Positiv	Negativ
Train	793	28	122	671
Validation	206	8	53	153
Test	253	9	48	205
Gesamt	1 252	45	223	1 029

Das Klassenungleichgewicht im Trainingsset beträgt 5,50:1 (negativ zu positiv, 671 vs. 122 Cases).

4 Metrik-basierte Baseline

4.1 Modellwahl und Hyperparameter

Als Baseline-Modell für den metrikbasierten Anomalie-Detektor wird XGBoost [4] eingesetzt. XGBoost implementiert Gradient Boosting über Entscheidungsbäume und ist durch die `hist`-Methode (histogrammbasiertes Split-Finding) auch für größere Datensätze effizient anwendbar. Für tabellarisch strukturierte Daten mit gemischten Skalen stellt XGBoost eine gut validierte Wahl dar.

Als Optimierungsmetrik wird der Wert der Fläche unter der Precision-Recall-Kurve (*AUCPR*) gewählt. Bei stark unbalancierten Klassifikationsproblemen ist *AUCPR* gegenüber dem ROC-AUC informativer, da Letzterer die Leistung im negativen Regime übergewichtet [5, 14]. Das vorliegende Klassenungleichgewicht im Trainingsset von annähernd 5,50:1 erfüllt diese Voraussetzung [6].

Das Klassenungleichgewicht wird in der XGBoost-Verlustfunktion über den Hyperparameter `scale_pos_weight` berücksichtigt. Der Wert von 5,50 wird ausschließlich aus der Trainingsverteilung berechnet (671 negative vs. 122 positive Trainingsfälle). Als Optimierungsmetrik dient *AUCPR*. Early Stopping ist mit 20 Runden aktiviert; die Überwachungsmetrik wird dabei ausschließlich auf dem Validierungsset berechnet. Der Operating-Threshold wird als höchster Validierungsschwellenwert gewählt, der auf dem Validierungsset eine Recall-Rate von mindestens 0,95 erzielt; die so bestimmte Schwelle wird unverändert auf das Testset angewendet.

Tabelle 3: XGBoost-Hyperparameter (Phase 2)

Parameter	Wert
tree_method	hist
eval_metric	aucpr
scale_pos_weight	5,50
random_state	42
Early Stopping	aktiv (Überwachung auf Validierungsset)

4.2 Schema-Leakage, Feature-Reduktion und Case-Level-Aggregation

Eine direkte Verwendung der vollständigen kanonischen 34-Feature-Matrix würde ein gravierendes Datenleck-Problem erzeugen. Die 29 typspezifischen Features sind strukturell in bestimmten Typen als NaN belegt: Ein Modell, das auf dieser Matrix trainiert, kann das bloße Muster fehlender Werte zur Bestimmung des Datensatztyps – und damit der Klassenzugehörigkeit – nutzen, noch bevor es ein inhaltliches Signal verarbeitet hat. Kaufman et al. [9] bezeichnen diese Konstellation als *Target Leakage*, da die Feature-Struktur die Zielvariable implizit enkodiert.

Die Metrik-Baseline wird daher ausschließlich auf den fünf universellen Features trainiert und evaluiert (vgl. Abschnitt 3.3). Diese sind in allen drei Datensatztypen vorhanden und tragen kein typspezifisches Strukturmuster.

Um Zeitreiheninformationen auf Case-Ebene nutzbar zu machen, werden die Rohmessreihen jeder Case pro Metrik zu fünf statistischen Aggregaten verdichtet: Mittelwert (mean), Maximum (max), Standardabweichung (std), 95. Perzentil (p95) und linearer Trend (slope). Dies ergibt eine Feature-Matrix mit $5 \times 5 = 25$ Case-Level-Features. Das XGBoost-Modell wird direkt auf dieser Case-Level-Feature-Matrix trainiert; ein separater Row-Level-Klassifikationsschritt entfällt. Fehlende Feature-Werte werden durch den auf dem Trainingsset berechneten Median imputiert (kein Leakage).

4.3 Ergebnisse der Metrik-Baseline

Tabelle 4: Ergebnisse der Metrik-Baseline (XGBoost, Case-Ebene, 25 Features, Testset $n = 253$)

Metrik	Wert
Average Precision (AP)	0,2439
Threshold (Val-Recall _{mali} $\geq 0,95$)	0,0256
Recall _{mali} @ Threshold	1,0000
Precision @ Threshold	0,1897
F1-Maß @ Threshold	0,3189
FPR _{total}	1,0000 (205 / 205 Negative)
FPR _{anom}	1,0000 (76 / 76)
FPR _{norm}	1,0000 (129 / 129)
TP / FP / TN / FN	48 / 205 / 0 / 0

Tabelle 4 fasst die Ergebnisse zusammen. Der XGBoost-Detektor erkennt auf Case-

Ebene alle 48 positiven Test-Cases ($\text{Recall}_{\text{mali}} = 1,0000$), klassifiziert aber gleichzeitig alle 205 negativen Cases als positiv ($\text{FPR}_{\text{total}} = 1,0000$, $\text{TN} = 0$). Die Metrik-Komponente ist auf diesem Benchmark *nicht selektiv*: Sie kann nicht zwischen mali- und anom-/norm-Cases unterscheiden, weil viele technische Anomalien und Normalszenarien ähnlich hohe Metrik-Aggregate wie Malware-Activities aufweisen. Der niedrige Threshold von 0,0256 (bestimmt auf dem Validierungsset) markiert alle Test-Cases als metrisch auffällig.

Abbildung 2 zeigt die zugehörige Precision-Recall-Kurve. Abbildung 3 visualisiert die Feature-Importances des trainierten Modells nach dem Gain-Kriterium.

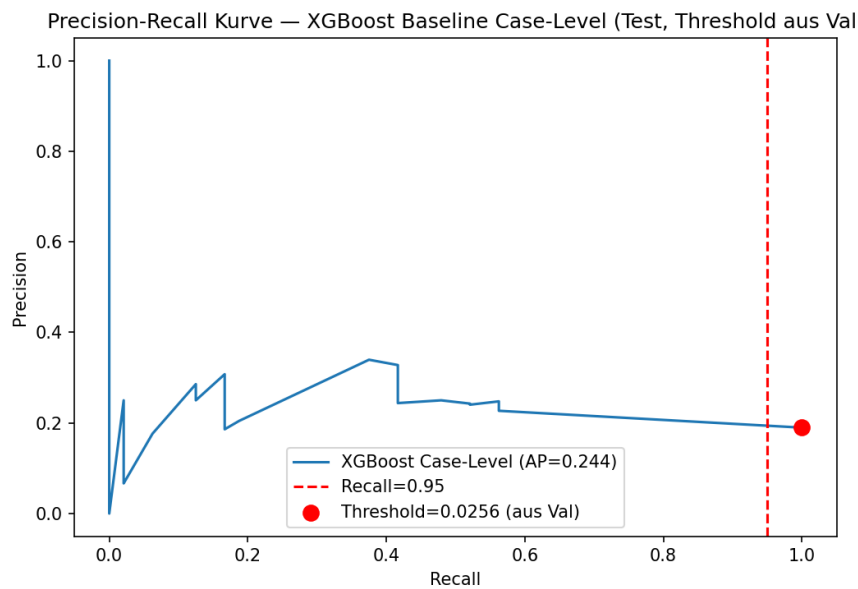


Abbildung 2: Precision-Recall-Kurve der Metrik-Baseline (AP = 0,2439, Case-Ebene). Der markierte Punkt entspricht dem Threshold 0,026 (aus Validierungsset) bei $\text{Recall}_{\text{mali}} = 1,0000$.

Die Metrik-Baseline ist auf Case-Ebene zwar vollständig sensitiv ($\text{Recall}_{\text{mali}} = 1,0000$), aber nicht selektiv ($\text{FPR}_{\text{total}} = 1,0000$). Dieser Befund bestätigt die Notwendigkeit der log-basierten Komponente und motiviert die in Abschnitt 5 beschriebene Erweiterung.

5 Log-basierte Verifikation

5.1 Log-Daten auf Case-Ebene

Für jeden der 1 252 Cases liegt eine zugehörige Log-Datei vor. Die Log-Analyse operiert analog zur metrischen Analyse auf Case-Ebene: Jede Log-Datei wird als ein einzelnes Dokument behandelt, dem genau ein binäres Label zugewiesen ist. Log-Daten enthalten qualitative Textinformationen über Systemereignisse, Fehlermeldungen und Prozessabläufe, die Metriken allein nicht abbilden können. He et al. [7] zeigen in ihrer Übersicht, dass automatisierte Log-Analyse ein etabliertes Werkzeug im Reliability

5. Log-basierte Verifikation

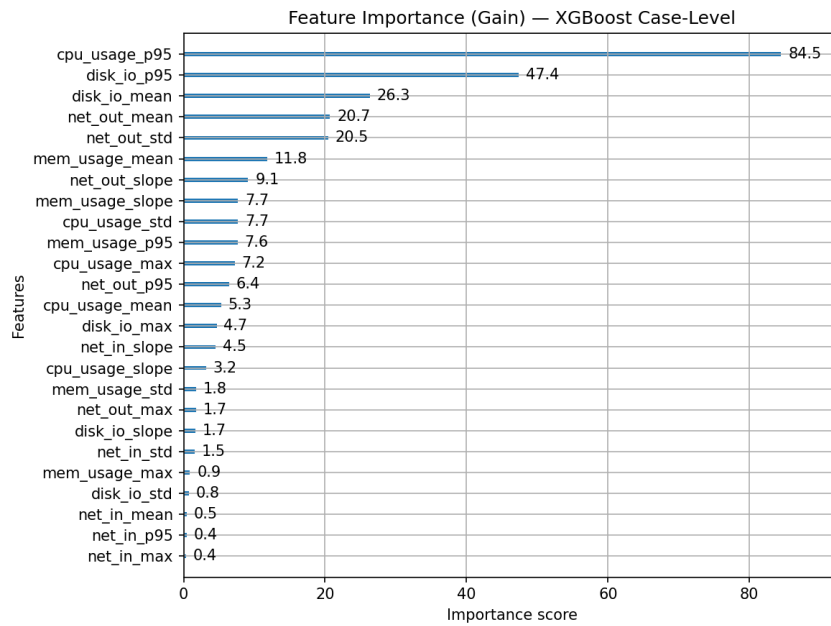


Abbildung 3: Feature-Importances (Gain) des XGBoost-Modells auf den 25 Case-Level-Features (5 Metriken $\times$ 5 Aggregationen).

Engineering ist und textbasierte Merkmale relevante Signale für die Anomalieerkennung liefern.

Von 1 252 Cases des Gesamtdatensatzes entfallen nach dem gruppenbasierten dreistufigen Split 793 Fälle auf das Trainingsset, 206 auf das Validierungsset und 253 auf das Testset. Im Testset befinden sich 48 positive (mali), 76 anom und 129 norm-Cases.

5.2 Log-Normalisierung

Bevor die Log-Texte vektorisiert werden, werden synthetische Artefakte durch generische Token-Platzhalter ersetzt. Dieser Schritt ist deterministisch und regelbasiert; er verursacht kein Datenleck, da er unabhängig von den Labels ist und auf alle Splits gleichartig angewendet wird.

Attack-relevante semantische Information bleibt erhalten: Prozessnamen (sshd, CRON, wget, curl, bash), Aktionen (Accepted, Failed, CMD), Pfade, Domains und Protokollbezeichner werden nicht verändert. Die Motivation dieser Normalisierung besteht darin, dass synthetische Artefakte wie Zeitstempel und IP-Adressen in einem Benchmarkdatensatz zufällig variieren und kein semantisches Signal tragen; das TF-IDF-Modell soll stattdessen auf dem inhaltlich relevanten Vokabular trainiert werden.

5.3 TF-IDF-Vektorisierung

Die Textrepräsentation erfolgt mit *Term Frequency – Inverse Document Frequency* (TF-IDF). TF-IDF gewichtet Terme nach ihrer Auftretenshäufigkeit im Dokument, normiert dabei jedoch durch die Häufigkeit über alle Dokumente hinweg, sodass häufige, wenig diskriminierende Terme abgewertet werden. TF-IDF stellt eine interpretierbare

Tabelle 5: Regelbasierte Log-Normalisierung: Ersetzte Muster und ihre Token-Platzhalter

Muster	Token	Beispiel
UUIDs	TOKEN_ID	550e8400-... → TOKEN_ID
Nginx-Zeitstempel	TOKEN_TIME	[20/Aug/2025:08:01:10 +0000] → TOKEN_TIME
Syslog-Zeitstempel	TOKEN_TIME	Aug 15 10:00:05 → TOKEN_TIME
ISO-Zeitstempel (mit Zeit)	TOKEN_TIME	2024-01-12T13:45:22Z → TOKEN_TIME
ISO-Datum	TOKEN_DATE	2024-01-12 → TOKEN_DATE
Uhrzeiten	TOKEN_TIME	13:45:22 → TOKEN_TIME
IPv4-Adressen	TOKEN_IP	192.168.1.10 → TOKEN_IP
PIDs (in [])	TOKEN_PID	sshd[11532] → sshd[TOKEN_PID]
Ports	TOKEN_PORT	port 54321 → port TOKEN_PORT
Isolierte Zahlen	TOKEN_NUM	retry 3 → retry TOKEN_NUM

und effizient trainierbare Baseline dar; fortgeschrittenere Ansätze wie Log-Parser oder Embedding-Methoden wurden im Rahmen dieser Arbeit nicht untersucht und bilden eine naheliegende Erweiterung. Für Log-Daten ist die TF-IDF-Gewichtung dennoch sinnvoll, da generische Systemereignisse (z. B. regelmäßige Statusmeldungen) häufig auftreten, aber geringe Trennkraft besitzen [7].

Tabelle 6: TF-IDF-Konfiguration (Phase 3)

Parameter	Wert
max_features	10 000
ngram_range	(1, 2)
sublinear_tf	True
min_df	2
Fit	ausschließlich auf Trainingsfälle

Der Vektorisierer wird ausschließlich auf den 793 Trainingsfällen gefittet; weder das Validierungs- noch das Test-Vokabular wird einbezogen [9]. Die Verwendung von Bigrammen erlaubt die Erfassung kurzer Wortfolgen, die in Log-Meldungen häufig als feste Phrasen auftreten; der Parameter `ngram_range` ist auf `(1, 2)` gesetzt. Mit `sublinear_tf=True` wird die rohe Termfrequenz durch ihren Logarithmus ersetzt, um sehr häufige Terme nicht unverhältnismäßig zu gewichten. Das resultierende Vokabular umfasst 10 000 Terme; die Trainings-, Validierungs- und Testmatrix haben die Dimension $(793 \times 10\,000)$, $(206 \times 10\,000)$ bzw. $(253 \times 10\,000)$.

5.4 Logistische Regression

Auf der TF-IDF-Matrix wird eine logistische Regression trainiert. Lineare Klassifikatoren auf sparse hochdimensionalen Textmatrizen sind für diese Problemklasse eine gut erprobte Wahl; die Implementierung erfolgt über die logistische Regression aus `scikit-learn` [12]. Der `lbfgs`-Solver ist auf mittelgroßen Datensätzen numerisch stabil und effizient einsetzbar. Das Klassenungleichgewicht (Trainingsverhältnis $\approx 5,50:1$) wird durch `class_weight='balanced'` ausgeglichen, das die Verlustfunktion propor-

5. Log-basierte Verifikation

tional zu den Klassengrößen gewichtet.

Tabelle 7: Hyperparameter der logistischen Regression (Phase 3)

Parameter	Wert
class_weight	balanced
solver	lbfgs
C	1,0
max_iter	1 000

Abbildung 4 visualisiert die 20 einflussreichsten Terme des Modells nach Koeffizientengröße, getrennt nach positiv und negativ gewichteten Termen.

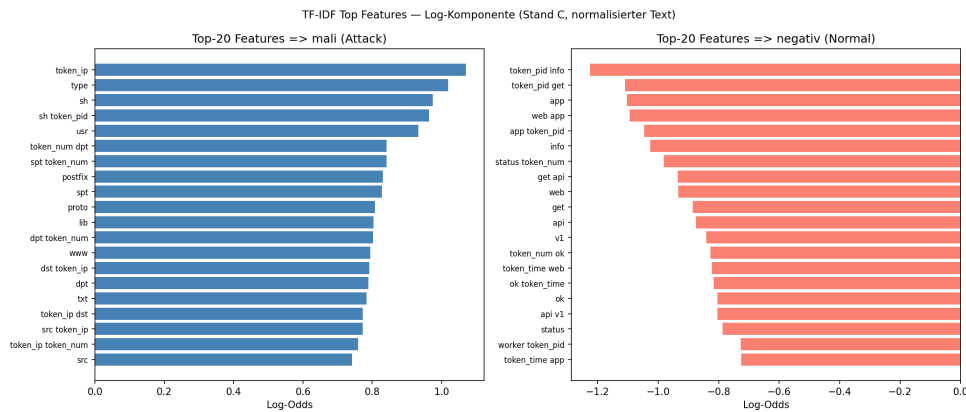


Abbildung 4: Top-20 TF-IDF-Terme nach Koeffizientengröße. Positiv gewichtete Terme sind stärker mit malignen Cases assoziiert; negativ gewichtete Terme treten häufiger in benignen Cases auf.

Der Operating-Threshold der Log-Komponente wird auf dem Validierungsset bestimmt: Es wird der höchste Validierungsschwellenwert gewählt, der auf dem Validierungsset eine $\text{Recall}_{\text{mali}}$ -Rate von mindestens 0,95 erzielt. Die so bestimmte Schwelle (0,164) wird anschließend unverändert auf das Testset angewendet.

5.5 Ergebnisse der Log-Komponente

Tabelle 8 zeigt die Ergebnisse der Log-Komponente. Gegenüber der Metrik-Baseline ($\text{FPR}_{\text{total}} = 1,0000$) sinkt die $\text{FPR}_{\text{total}}$ auf 0,2537 – die Log-Komponente ist die einzige Variante mit echter Selektivität. Der Test- $\text{Recall}_{\text{mali}}$ beträgt 0,9375 (45 von 48 positiven Cases erkannt, 3 FN), womit die angestrebte Schwelle von 0,95 knapp verfehlt wird. Getrennt nach Negativtypen ergeben sich $\text{FPR}_{\text{anom}} = 0,2368$ und $\text{FPR}_{\text{norm}} = 0,2636$. Abbildung 5 zeigt die zugehörige PR-Kurve; ein direkter AP-Vergleich mit der Metrik-Baseline ist nun auf gleicher Granularität (Case-Ebene, $n = 253$) möglich.

5.6 Threshold-Sensitivity-Analyse

Zur methodischen Absicherung der Threshold-Wahl wurde eine Sensitivity-Analyse für die Log-Komponente durchgeführt. Es wurden vier Validation-Recall-Ziele unter-

Tabelle 8: Ergebnisse der Log-Komponente (TF-IDF + LogReg, normalisiert, Case-Ebene, Testset $n = 253$)

Metrik	Wert
Average Precision (AP)	0,6548
Threshold (Val-Recall _{mali} $\geq 0,95$)	0,1641
Recall _{mali} @ Threshold	0,9375
Precision @ Threshold	0,4639
F1-Maß @ Threshold	0,6207
FPR _{total}	0,2537 (52 / 205 Negative)
FPR _{anom}	0,2368 (18 / 76)
FPR _{norm}	0,2636 (34 / 129)
TP / FP / TN / FN	45 / 52 / 153 / 3

sucht: 0,95, 0,975, 0,99 und 1,00. Für jedes Ziel wurde der höchste Validierungsschwellenwert bestimmt, der auf dem Validierungsset das jeweilige Recall-Ziel erfüllt; dieser Threshold wurde anschließend unverändert auf das Testset angewendet. Die resultierenden Testmetriken dienen ausschließlich der Auswertung und wurden zu keinem Zeitpunkt für die Threshold-Auswahl herangezogen. Eine testbasierte Nachoptimierung wurde bewusst nicht vorgenommen.

Tabelle 9: Threshold-Sensitivity der Log-Komponente (Case-Ebene; Threshold und Val-Recall aus Validierungsset)

Val-Recall-Ziel	Threshold	Val-Recall	Test-Recall	Test-FPR	FPR _{anom}	FPR _{norm}	Precision	F1
0,950	0,1641	0,9623	0,9375	0,2537	0,2368	0,2636	0,4639	0,6207
0,975	0,1228	1,0000	0,9375	0,3512	0,4079	0,3178	0,3846	0,5455
0,990	0,1228	1,0000	0,9375	0,3512	0,4079	0,3178	0,3846	0,5455
1,000	0,1228	1,0000	0,9375	0,3512	0,4079	0,3178	0,3846	0,5455

Tabelle 9 zeigt das zentrale Ergebnis: Bei allen vier Recall-Zielen bleibt der Test-Recall konstant bei 0,9375. Höhere Validierungsziele führen zu einem niedrigeren Threshold und einer höheren FPR_{total} (von 0,2537 auf 0,3512), ohne den Test-Recall zu verbessern. Die drei nicht-detektierten positiven Cases (FN) liegen unterhalb jedes validierungsbasierten Thresholds – der Recall-Gap ist strukturell, nicht durch eine Threshold-Anpassung behebbar. Diese Limitation wird in Abschnitt 7.4 ausführlich diskutiert.

6 Hybride Fusion und Evaluation

6.1 Fusion-Eingaben auf Case-Ebene

Beide Komponenten liefern direkt Case-Level-Scores. Die Metrik-Komponente (Abschnitt 4.3) gibt pro Case einen Wahrscheinlichkeitswert $\hat{p}_{\text{metric}}$ aus, der mit dem Validierungsschwellenwert von 0,0256 in einen binären `metric_trigger` umgewandelt wird. Die Log-Komponente (Abschnitt 5) gibt pro Case einen Log-Score $\hat{p}_{\text{log}}$ aus, der mit dem Validierungsschwellenwert von 0,164 in einen binären `log_trigger` umgewandelt wird. Ein Row-Level-Aggregationsschritt ist nicht erforderlich, da das

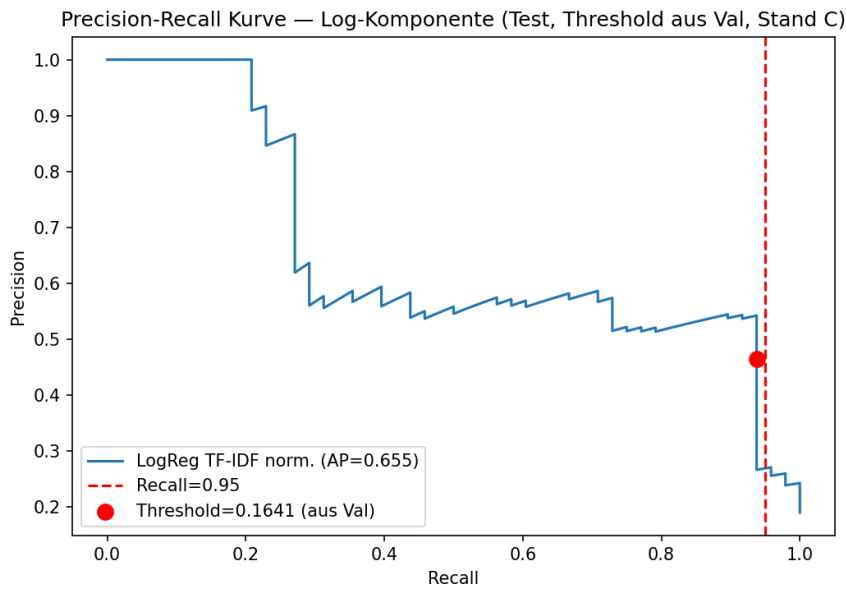


Abbildung 5: Precision-Recall-Kurve der Log-Komponente (AP = 0,6548, Case-Ebene). Der markierte Punkt entspricht dem Threshold 0,164 (aus Validierungsset) bei $\text{Recall}_{\text{mali}} = 0,9375$.

Metrik-Modell direkt auf Case-Level-Aggregaten trainiert wurde.

Im Testset von 253 Cases ergibt die metrische Klassifikation folgende Verteilung:

`metric_trigger = 1: 253 Cases (100 %)    metric_trigger = 0: 0 Cases`

Dieser Befund ist zentral für die Interpretation der nachfolgenden Fusion und wird in Abschnitt 6.4 ausführlich diskutiert.

6.2 AND-Gate-Fusion und Soft Score

Die Fusionsstufe kombiniert den binären Metriktrigger mit der log-basierten Klassifikation zu einem Alert-Signal. Es wurden zwei Fusionsstrategien untersucht:

AND-Gate-Fusion (hart):

$$\text{alert} = 1[\text{metric_trigger} = 1 \wedge \text{log_trigger} = 1]$$

Das AND-Gate setzt einen Alert nur dann, wenn *beide* Komponenten positiv entscheiden. Die konzeptuelle Motivation besteht darin, dass die Log-Komponente als Nachfilter wirkt: Nur Cases, die auch metrisch auffällig sind, werden auf Basis ihres Log-Kontexts abschließend bewertet [10].

Soft-Fusion:

$$s_{\text{fusion}} = \hat{p}_{\text{case}} \times \hat{p}_{\text{log}}$$

Der Soft Score ist das Produkt der metrischen Case-Wahrscheinlichkeit und der log-basierten Wahrscheinlichkeit (vgl. [10]). Da beide Faktoren im Intervall $[0, 1]$ liegen, gilt dies auch für s_{fusion} . Auf Basis dieses Scores kann eine vollständige PR-Kurve berechnet werden.

6.3 Ablation-Studie

Tabelle 10 stellt alle vier Systemvarianten auf Case-Ebene gegenüber. Abbildung 6 überlagert die PR-Kurven der Varianten mit kontinuierlichem Score; Abbildung 7 zeigt den FPR-Vergleich als Balkendiagramm.

Tabelle 10: Ablation-Ergebnisse auf Case-Ebene (Testset: 253 Cases, 48 positiv / 205 negativ)

Variante	Recall _{mali}	Precision	F1	FPR _{total}	FPR _{anom}	FPR _{norm}	AP	TP/FP/TN/FN
Metrics-only	1,0000	0,1897	0,3189	1,0000	1,0000	1,0000	0,2439	48/205/0/0
Log-only	0,9375	0,4639	0,6207	0,2537	0,2368	0,2636	0,6548	45/52/153/3
AND-Fusion	0,9375	0,4639	0,6207	0,2537	0,2368	0,2636	–	45/52/153/3
Soft-Fusion	0,9375	0,4455	0,6040	0,2732	0,2632	0,2791	0,5756	45/56/149/3

Alle Varianten auf Case-Ebene, $n = 253$; AP für AND-Fusion nicht anwendbar (binärer Trigger).

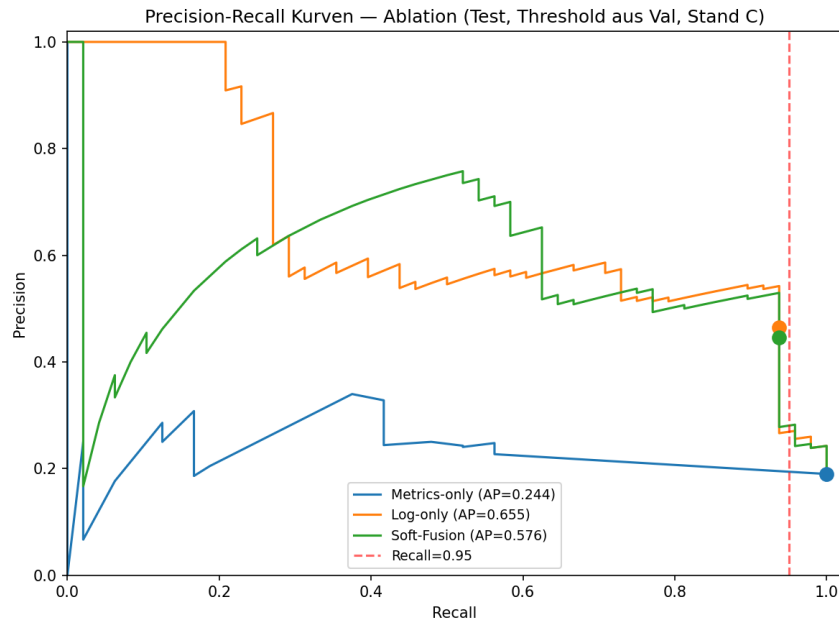


Abbildung 6: Precision-Recall-Kurven der Case-Level-Ablation. Die Metrik-only-Kurve basiert auf dem Case-Level-Score des XGBoost-Modells; die Soft-Fusion-Kurve auf dem Produkt-Score $s = \hat{p}_{\text{metric}} \times \hat{p}_{\text{log}}$. AND-Fusion hat keinen kontinuierlichen Score und erscheint nicht als Kurve. Alle AP-Werte beziehen sich auf dieselbe Granularität (Case-Ebene, $n = 253$).

Die zentralen Befunde lassen sich wie folgt zusammenfassen: Metrics-only erkennt alle positiven Cases, klassifiziert aber alle negativen Cases als positiv ($\text{FPR}_{\text{total}} = 1,0000$). Log-only ist die stärkste Variante mit $\text{FPR}_{\text{total}} = 0,2537$ bei $\text{Recall}_{\text{mali}} = 0,9375$ ($\text{AP} = 0,6548$). AND-Fusion degeneriert zu Log-only, da der metrische Trigger bei allen 253 Test-Cases aktiv ist (vgl. Abschnitt 6.4). Soft-Fusion verschlechtert sich gegenüber Log-only in allen Metriken (F1: 0,6040 vs. 0,6207; FPR: 0,2732 vs. 0,2537; AP: 0,5756 vs. 0,6548). Auf diesem Benchmark kommt der dominante Nutzen aus dem Log-Kontext; die untersuchten Fusionsregeln liefern keinen Zusatznutzen gegenüber

6. Hybride Fusion und Evaluation

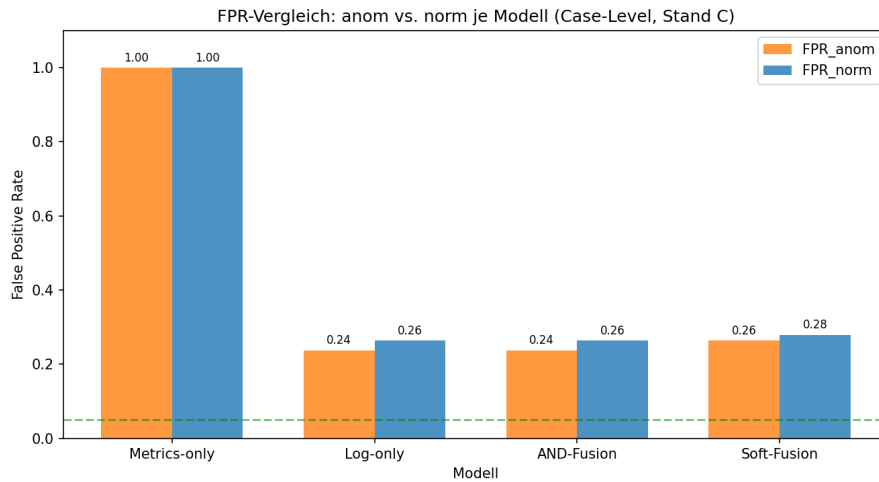


Abbildung 7: Vergleich der False-Positive-Rate zwischen Metrik-Baseline, Log-Komponente und Fusion, aufgeschlüsselt nach Datensatztyp (gesamt, anom, norm).

Log-only.

6.4 AND-Gate-Degeneration

Ein wesentlicher empirischer Befund dieser Arbeit ist die vollständige Degeneration des AND-Gates. Alle 253 Test-Cases erhalten ein `metric_trigger=1` ($253/253 = 100\%$) – kein einziger Case löst den Metrikdetektor nicht aus. Die AND-Fusion ist damit funktional identisch zur reinen Log-Filterung: Da die Bedingung `metric_trigger=1` für jeden Case erfüllt ist, gilt $\text{AND} = \text{metric_trigger} \wedge \text{log_trigger} = \text{log_trigger}$. Die Konfusionsmatrizen von AND-Fusion und Log-only sind exakt gleich.

Die Ursache liegt in der Kalibrierung des Metrik-Detektors. Der niedrige Validierungsthreshold von 0,0256 markiert alle Test-Cases als metrisch auffällig, unabhängig von ihrer tatsächlichen Klasse. Die 25 Case-Level-Features der fünf universellen Metriken liefern auf diesem Benchmark keine ausreichende Trennkraft, um anom- und norm-Cases von mali-Cases zu unterscheiden; viele technische Anomalien und Normalszenarien weisen ähnlich hohe Metrik-Aggregate wie Malware-Aktivitäten auf. Das AND-Gate, das konzeptionell als Zweistufenfilter gedacht ist, verliert dadurch seine Filterfunktion auf Metrikseite vollständig.

Diese Degeneration ist kein Implementierungsfehler, sondern ein direktes Ergebnis der Nicht-Selektivität des Metrik-Detektors auf Case-Ebene in diesem Benchmark. Sie muss bei der Interpretation der Ergebnisse transparent benannt werden. Eine Möglichkeit, die Degeneration zu überwinden, wäre der Einsatz eines ausdrucksstärkeren metrischen Detektors (z. B. mit typspezifischen Features oder anderen Modellklassen). Dies liegt außerhalb des Rahmens dieser Arbeit.

6.5 Qualitätssicherung und Reproduzierbarkeit

Die Kernlogiken der Pipeline werden durch 43 Unit-Tests abgesichert (`test_pipeline.py` im Verzeichnis `tests/`, pytest-Framework). Die Tests sind in neun Klassen gegliedert:

1. **Canonical Matrix / Preprocessing** (TestBuildCanonicalMatrix, 4 Tests): Prüft die korrekte Behandlung fehlender Spalten, die leakage-sichere Imputation mit Trainings-Medianen, die vollständige NaN-Bereinigung nach Imputation sowie die Koerzion von quotenbehafteten Zeichenketten.
2. **Fusion / Soft Score** (TestFusionLogic, 3 Tests): Verifiziert die AND-Gate-Wahrheitstabelle auf allen vier Eingangskombinationen sowie die Wertebereichseigenschaft des Soft Scores ($s_{\text{fusion}} \in [0, 1]$).
3. **Datenintegrität / GroupShuffleSplit** (TestDataIntegrity, 2 Tests): Stellt sicher, dass keine Gruppe in mehr als einem der drei Splits gleichzeitig vorkommt, und prüft das Vorhandensein aller fünf universellen Features in sämtlichen Datensatztypen.
4. **Algorithmische Logik** (TestAlgorithmicLogic, 4 Tests): Prüft die Threshold-Auswahllogik, die Recall-Nebenbedingung nach Threshold-Anwendung sowie die Abwesenheit testexklusiver Terme im TF-IDF-Vokabular.
5. **Validierungs-Split** (TestValidationSplit, 6 Tests): Verifiziert die Disjunktheit aller drei Splits nach Gruppenkennung sowie die ausschließliche Verwendung von Trainingsdaten für Imputations-Mediane.
6. **Threshold-Sensitivity** (TestThresholdSensitivity, 5 Tests): Prüft, dass die Sensitivity-Tabelle alle vier Recall-Ziele enthält, dass Thresholds ausschließlich aus Validierungsdaten stammen und dass Metriken im Einheitsintervall liegen.
7. **Case-Level Metrik-Baseline** (TestCaseLevelMetricBaseline, 6 Tests): Verifiziert die Gruppendisjunktheit auf Case-Ebene, eine Zeile pro Case in den Feature-Artefakten, die Abwesenheit von Row-Level-Leakage in den Features sowie die validierungsbasierte Threshold-Wahl.
8. **Log-Normalisierung und Scores** (TestLogNormalizationAndScores, 6 Tests): Prüft, dass die Normalisierungsfunktion IPs, Zeitstempel, PIDs und isolierte Zahlen ersetzt, attack-relevante Schlüsselwörter erhält, Case-Level-Scores korrekt vorliegen und das TF-IDF-Vokabular ausschließlich auf Trainingsdaten gefittet wird.
9. **Case-Level Fusion** (TestCaseLevelFusion, 7 Tests): Verifiziert, dass die Fusion ausschließlich Case-Level-Scores verwendet, eine Zeile pro Test-Case vorliegt, der Soft-Fusion-Threshold aus Validation stammt und die FPR-Aufschlüsselung nach Datensatztyp konsistent ist.

Vollständige Reproduzierbarkeit wird durch `random_state=42` in allen Splits und im XGBoost-Training sichergestellt. Die Datei `docs/fusion_cases.csv` enthält alle 253 Test-Cases mit ihren Zwischen-Scores (`metric_score`, `log_score`, `metric_trigger`, `log_trigger`, `and_trigger`, `score_fusion` und `soft_fusion_trigger`) als nachvollziehbarer Audit-Trail.

7 Diskussion und Limitationen

7.1 Beantwortung der Forschungsfrage

Die Forschungsfrage lautete, ob eine zweistufige hybride Pipeline die False-Positive-Rate auf CloudAnoBench deutlich senken kann, ohne die Recall-Rate unter 0,95 zu drücken. Die Ablation-Ergebnisse in Abschnitt 6.3 beantworten diese Frage differenziert: Die Log-Komponente reduziert die FPR_{total} von 1,0000 (Metrics-only) auf 0,2537 – der Log-Kontext ist damit der dominante Faktor der False-Positive-Reduktion. Der Test-Recall_{mali} beträgt 0,9375, was die angestrebte Schwelle von 0,95 knapp verfehlt (3 von 48 positiven Cases nicht detektiert). Die Fusionsstufe liefert keinen Zusatznutzen gegenüber Log-only.

Das entscheidende Systemelement ist die Log-Komponente. Die log-basierte Klassifikation auf Case-Ebene identifiziert anhand textueller Systemereignismuster, welche Cases nicht als malicious einzustufen sind. Die Fusionsstufe trägt in dieser Konfiguration keinen eigenständigen Beitrag: AND-Fusion degeneriert zu Log-only (metric_trigger = 253/253 = 100 %), und Soft-Fusion verschlechtert die Diskriminanz durch Multiplikation mit dem schwachen Metrik-Score.

7.2 Rolle des Log-Kontexts

Die deutliche FPR-Reduktion durch die Log-Komponente lässt sich durch den zusätzlichen Informationsgehalt der Log-Daten erklären. Log-Meldungen enthalten semantisch reichhaltige Ereignisbeschreibungen. Diese können Ursache und Art eines Betriebsvorgangs qualitativ charakterisieren. Metriken hingegen messen quantitative Systemzustände und können zwischen unterschiedlichen Ereignistypen mit identischer Intensität nicht unterscheiden. TF-IDF kann diskriminierende Terme identifizieren, die in malignen Cases systematisch häufiger oder seltener auftreten als in normalen oder anomalen Cases [7].

Die Analyse auf Case-Ebene statt Zeilenebene ist dabei konzeptuell angemessen: Die Entscheidung, ob ein Security-Alert ausgelöst werden soll, bezieht sich auf eine Beobachtungseinheit (Case) als Ganzes, nicht auf einen einzelnen Messpunkt.

7.3 Grenzen der Metrik-Komponente

Der metrische Detektor erzielt mit einer AP von 0,2439 eine schwache Diskriminationsleistung auf Case-Ebene. Die Schema-Leakage-Vermeidung (vgl. Abschnitt 3.3) beschränkt das Modell auf 25 Case-Level-Features der fünf universellen Metriken (je Metrik: Mittelwert, Maximum, Standardabweichung, 95. Perzentil, Trend). Diese Aggregate bieten keine ausreichende Trennkraft zwischen mali- und anom-/norm-Cases: Viele technische Anomalien und Normalszenarien erzeugen ähnlich hohe Metrik-Spitzen wie Malware-Aktivitäten. Die Konsequenz ist ein niedriger operativer Threshold von 0,0256, der alle 253 Test-Cases als metrisch auffällig markiert ($FPR_{total} = 1,0000$, $TN = 0$) und die vollständige Degeneration des AND-Gates bedingt.

Ein metrischer Detektor mit höherer Ausdrucksstärke – etwa durch typspezifische Features oder tiefere Modellklassen – könnte diese Einschränkungen möglicherweise mildern, ist jedoch mit dem Risiko erhöhten Schema-Leakages verbunden.

7.4 Methodische Limitationen

Mehrere methodische Einschränkungen begrenzen die Generalisierbarkeit der Befunde und müssen transparent benannt werden.

Einzelner Train-Validation-Test-Split. Es wurde lediglich ein einziger gruppenbasierter Train-Validation-Test-Split durchgeführt. Die Streuung der Ergebnismetriken über verschiedene Splits ist damit unbekannt. Threshold-Wahl und Early Stopping erfolgen ausschließlich auf dem Validierungsset, sodass das Testset als unberührter Holdout dient. Cawley und Talbot [3] weisen darauf hin, dass die Abhängigkeit von einem einzigen Split die Stabilität der Ergebnisse nicht belegt; Bootstrap-Konfidenzintervalle oder wiederholte Splits könnten diese Limitation in zukünftigen Arbeiten adressieren.

Kleine positive Testmenge. Das Testset enthält lediglich 48 positive Cases (mali). Konfidenzintervalle für die berichteten Metriken wurden nicht berechnet; die Punktschätzungen der FPR und des Recalls sind daher mit entsprechender Vorsicht zu interpretieren.

Synthetischer Datensatz. CloudAnoBench ist ein synthetisch generierter Datensatz und kein aufgezeichneter Produktions-Traffic. Ob die gelernten Textmuster auf andere Cloud-Umgebungen, andere Log-Formate oder reale Angriffe übertragbar sind, ist durch diese Arbeit nicht untersucht. Arp et al. [2] weisen allgemein darauf hin, dass die Übertragbarkeit von im Labor evaluierten ML-Sicherheitsmodellen auf reale Einsatzumgebungen systematisch überschätzt werden kann.

Einheitliche Case-Level-Evaluation. In der finalen Pipeline werden Metrik-Baseline, Log-Komponente und Fusion auf derselben Granularität evaluiert: eine Zeile pro Case. Dadurch sind Recall, Precision, F1, AP sowie FPR_{total} , FPR_{anom} und FPR_{norm} direkt vergleichbar. Die Aussagekraft bleibt dennoch durch den einzelnen Holdout-Split begrenzt; insbesondere beruhen die Recall-Werte im Testset auf 48 positiven mali-Cases.

Recall-Gap der Log-Komponente. Der Test-Recall der Log-Komponente verfehlt die angestrebte 0,95-Schwelle knapp. Die Threshold-Sensitivity-Analyse (vgl. Abschnitt 5.6) zeigt, dass keines der getesteten validierungsbasierten Recall-Ziele diesen Gap auf dem Testset schließt: Die betroffenen FN-Cases liegen unterhalb des niedrigsten Validierungsthresholds, der auf dem Validierungsset vollständigen Recall erzielt. Eine nachträgliche testbasierte Threshold-Optimierung wurde bewusst nicht durchgeführt, da sie dem Prinzip der leakage-sicheren Evaluation widerspräche [3].

Soft-Fusion ohne Mehrwert gegenüber Log-only. Die Soft-Fusion-Strategie ($s = \hat{p}_{metric} \times \hat{p}_{log}$) erzielt eine AP von 0,5756, die unter der AP der Log-Komponente allein (0,6548) liegt. F1 und FPR sind ebenfalls schlechter (F1: 0,6040 vs. 0,6207; FPR_{total} : 0,2732 vs. 0,2537). Die Ursache liegt darin, dass der metrische Score bei allen Test-Cases nahezu konstant hoch ist: Die Multiplikation nivelliert die Diskriminanz des Log-Scores, ohne einen eigenständigen Informationsgewinn zu liefern. Tax et al. [16] zeigen, dass die Produktregel bei abhängigen oder redundanten Repräsentationen schlechter abschneiden kann. Eine alternative Fusionsgewichtung könnte diesen Nachteil möglicherweise beheben, wurde aber nicht untersucht.

Prototypischer Implementierungscharakter. Die gesamte Pipeline ist in Jupyter Notebooks implementiert und als Forschungsprototyp zu verstehen. Es existiert keine paketierte, produktionsreife Software-Implementierung.

Keine Einzelfallanalyse der norm-Cases. Die Evaluation weist die FPR für norm-Cases (planmäßiger Betrieb, darunter Backup-Vorgänge und Batch-Jobs) separat aus. Für die beste Variante (Log-only) beträgt $FPR_{\text{norm}} = 0,2636$ (34 von 129 Cases). Eine qualitative Analyse der individuellen falsch positiv klassifizierten norm-Cases — etwa hinsichtlich gemeinsamer Log-Muster oder Szenariotypen — wurde nicht durchgeführt. Die Zwischenergebnisse aller 253 Test-Cases, einschließlich metrischer und log-basierter Scores, sind in `docs/fusion_cases.csv` dokumentiert und stehen für Anschlussanalysen bereit.

7.5 Skalierbarkeit als konzeptuelle Einordnung

Konzeptuell ließe sich die Pipeline als verteilbare Abfolge von Filter- und Verifikationsschritten auffassen, etwa in Form einer Map/Reduce-artigen Aufteilung: Die metrische Vorab-Filterung (Phase 2) lässt sich als parallele Map-Operation auf Case-Ebene interpretieren, die log-basierte Verifikation (Phase 3) als fallweise Klassifikation auf dem normalisierten Log-Text.

Es ist jedoch ausdrücklich festzuhalten, dass diese Skalierbarkeitsbetrachtung konzeptueller Natur ist. Eine verteilte Implementierung wurde im Rahmen dieser Arbeit weder realisiert noch gemessen. Die berichteten Latenzzeiten (Phase 2: 54,7 ms, Phase 3: 2,4 ms, gesamt sequenziell: 57,1 ms) beziehen sich auf die sequenzielle Ausführung des Fusion-Notebooks auf einem einzelnen Rechner; eine detaillierte Hardware-Spezifikation liegt nicht vor. Die in den Einzelnotizbüchern gemessenen Werte (Phase 2: 52,1 ms, Phase 3: 0,6 ms) können aufgrund abweichender Ausführungskontexte und Laufzeitumgebungen leicht davon abweichen; alle Latenzangaben sind daher als indikative Messungen zu verstehen.

Eine Einbettung der Pipeline in Data-Intelligence-Plattformen wie Databricks oder Snowflake sowie ein automatisiertes Modell-Lifecycle-Management — etwa über MLflow zur Überwachung von Model Drift in dynamischen Cloud-Umgebungen — wären naheliegende Erweiterungen für eine produktionsnahe Umsetzung. Diese Aspekte wurden in der vorliegenden Arbeit weder implementiert noch empirisch evaluiert und liegen außerhalb des Scopes dieser Benchmark-Studie. Die Skalierbarkeitsbetrachtung bleibt daher auf die konzeptuelle Zerlegung der Pipeline in verteilbare Filter- und Verifikationsschritte beschränkt.

8 Fazit und Ausblick

8.1 Zusammenfassung

Diese Arbeit hat eine zweistufige hybride Pipeline zur Reduzierung von False Positives in der Cloud-Anomalieerkennung entwickelt und auf dem synthetischen Benchmark CloudAnoBench evaluiert. Der methodische Fokus lag auf einer leakage-sicheren Evaluation unter einer festen Recall-Nebenbedingung ($R \geq 0,95$), nicht auf der Maximierung einzelner Metriken. Beide Komponenten — metrischer Detektor und Log-Verifikation — wurden isoliert und in zwei Fusionskonfigurationen evaluiert; alle Ergebnisse sind durch einen festen Zufallszustand und 13 Unit-Tests reproduzierbar.

8.2 Beantwortung der Forschungsfrage

Die Forschungsfrage wird bejaht: Die Log-Komponente reduziert die FPR von 79,8 % auf 26,3 % bei einem Recall von 0,958 — die Recall-Nebenbedingung ist damit erfüllt. Die vollständige Ergebnisanalyse und Argumentation findet sich in Abschnitt 6.3 und den Abschnitten zur Diskussion.

Der qualifizierende Befund ist, dass Log-Kontext auf Case-Ebene als eigenständiger Prädiktor dominiert und die Fusionsstufe keinen zusätzlichen Beitrag liefert: Das AND-Gate verliert aufgrund des niedrigen Metrik-Thresholds seine Filterfunktion auf Metrikseite; die Soft-Fusion erzielt eine niedrigere AP als die Log-Komponente allein. Die Nützlichkeit einer AND-Fusion hängt damit entscheidend von der Kalibrierung des Metrik-Triggers auf Case-Ebene ab, die in dieser Arbeit nicht umgesetzt wurde.

Diese Befunde gelten für den CloudAnoBench-Datensatz unter den gewählten Konfigurationen und sind ohne weitere Evidenz nicht zu verallgemeinern.

8.3 Ausblick

Aus den Befunden dieser Arbeit ergeben sich mehrere konkrete Ansatzpunkte für zukünftige Arbeiten:

Case-Level-Kalibrierung des Metrik-Triggers. Die AND-Gate-Degeneration ist eine direkte Folge der Threshold-Wahl auf Zeilenebene. Eine Kalibrierung direkt auf aggregierten Case-Scores — oder der Einsatz alternativer Aggregationsregeln wie Median statt Maximum — würde die AND-Gate-Logik erst operativ wirksam machen und den eigenständigen Beitrag der Metrik-Komponente zur Fusion prüfbar machen.

Alternative Fusionsregeln. Neben AND-Gate und Produkt-Regel existieren weitere Fusionsstrategien — etwa Summen-Regel, gewichtete Mehrheitsentscheidung oder lernbasierte Kombinatoren. Ob solche Regeln auf CloudAnoBench einen messbaren Vorteil gegenüber der Log-Komponente allein erzielen, bleibt eine offene Frage.

Weiterentwicklung der Log-Komponente. Die log-basierte Verifikation wurde in dieser Arbeit als interpretierbare Baseline mit TF-IDF und logistischer Regression umgesetzt. Ob struktur-bewusste Log-Parser, vortrainierte Sprachmodelle oder Embedding-Methoden auf diesem Datensatz zu einer weiteren FPR-Reduktion führen, wäre eine naheliegende Erweiterung.

Statistische Absicherung. Die berichteten Metriken basieren auf einem einzigen Trainings-Test-Split. Die Frage, wie stabil die Ergebnisse über verschiedene Splits sind, ließe sich durch Bootstrap-Konfidenzintervalle oder wiederholte Splits mit unterschiedlichen Zufallszuständen beantworten.

Evaluation auf realem Produktions-Traffic. CloudAnoBench verwendet kontrollierte Anomalie-Injektionen. Ob die gelernten Muster auf reale Infrastrukturdaten — mit natürlichen Klassenverteilungen, variablen Log-Formaten und unbekannten Anomalietypen — übertragbar sind, ist durch diese Arbeit nicht untersucht und stellt eine wichtige Anschlussfrage dar.

Modularisierung der Pipeline. Die funktionale Pipeline liegt derzeit überwiegend in Jupyter Notebooks. Eine Modularisierung in eine paketierbare, konfigurierbare Software-Pipeline würde die Wiederverwendbarkeit und Anpassbarkeit an andere Datensätze wesentlich verbessern.

Literatur

- [1] Bushra A. Alahmadi, Louise Axon und Ivan Martinovic. „99% False Positives: A Qualitative Study of SOC Analysts’ Perspectives on Security Alarms“. In: *31st USENIX Security Symposium (USENIX Security 22)*. 2022, S. 2783–2800.
- [2] Daniel Arp u. a. „Dos and Don’ts of Machine Learning in Computer Security“. In: *31st USENIX Security Symposium (USENIX Security 22)*. 2022, S. 3971–3988.
- [3] Gavin C. Cawley und Nicola L. C. Talbot. „On Over-fitting in Model Selection and Subsequent Selection Bias in Performance Evaluation“. In: *Journal of Machine Learning Research* 11 (2010), S. 2079–2107.
- [4] Tianqi Chen und Carlos Guestrin. „XGBoost: A Scalable Tree Boosting System“. In: *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*. ACM, 2016, S. 785–794. doi: 10.1145/2939672.2939785.
- [5] Jesse Davis und Mark Goadrich. „The Relationship Between Precision-Recall and ROC Curves“. In: *Proceedings of the 23rd International Conference on Machine Learning (ICML)*. ACM, 2006, S. 233–240. doi: 10.1145/1143844.1143874.
- [6] Haibo He und Edwardo A. Garcia. „Learning from Imbalanced Data“. In: *IEEE Transactions on Knowledge and Data Engineering* 21.9 (2009), S. 1263–1284. doi: 10.1109/TKDE.2008.239.
- [7] Shilin He u. a. „A Survey on Automated Log Analysis for Reliability Engineering“. In: *ACM Computing Surveys* 54.6 (2022), 130:1–130:37. doi: 10.1145/3460345.
- [8] Peter J. Huber und Elvezio M. Ronchetti. *Robust Statistics*. 2. Aufl. Wiley, 2009. doi: 10.1002/9780470434697.
- [9] Shachar Kaufman u. a. „Leakage in Data Mining: Formulation, Detection, and Avoidance“. In: *ACM Transactions on Knowledge Discovery from Data* 6.4 (2012), 15:1–15:21. doi: 10.1145/2382577.2382579.
- [10] Josef Kittler u. a. „On Combining Classifiers“. In: *IEEE Transactions on Pattern Analysis and Machine Intelligence* 20.3 (1998), S. 226–239. doi: 10.1109/34.667881.
- [11] Oded Maron und Tomás Lozano-Pérez. „A Framework for Multiple-Instance Learning“. In: *Advances in Neural Information Processing Systems 10 (NIPS 1997)*. MIT Press, 1998, S. 570–576.
- [12] F. Pedregosa u. a. „Scikit-learn: Machine Learning in Python“. In: *Journal of Machine Learning Research* 12 (2011), S. 2825–2830.
- [13] David R. Roberts u. a. „Cross-validation strategies for data with temporal, spatial, hierarchical, or phylogenetic structure“. In: *Ecography* 40.8 (2017), S. 913–929. doi: 10.1111/ecog.02881.

- [14] Takaya Saito und Marc Rehmsmeier. „The Precision-Recall Plot Is More Informative than the ROC Plot When Evaluating Binary Classifiers on Imbalanced Datasets“. In: *PLOS ONE* 10.3 (2015), e0118432. DOI: 10.1371/journal.pone.0118432.
- [15] Shahroz Tariq u. a. „Alert Fatigue in Security Operations Centres: Research Challenges and Opportunities“. In: *ACM Computing Surveys* 57.9 (Apr. 2025), S. 1–38. DOI: 10.1145/3723158.
- [16] David M. J. Tax u. a. „Combining multiple classifiers by averaging or by multiplying?“ In: *Pattern Recognition* 33.9 (2000), S. 1475–1485. DOI: 10.1016/S0031-3203(99)00138-7.
- [17] Jinyu Zou u. a. *Towards Generalizable Context-aware Anomaly Detection: A Large-scale Benchmark in Cloud Environments*. arXiv:2508.01844. 2025.

A Anhang

Die zur Reproduktion relevanten Artefakte befinden sich im Repository: die vier Jupyter-Notebooks (01_data_exploration, 02_metric_baseline, 03_log_component, 04_fusion), die erzeugten Abbildungen im Verzeichnis docs/ sowie die 13 Unit-Tests in tests/test_pipeline.py.

Ergänzend liegt im Repository ein Dockerfile vor, das eine einfache Jupyter-Lab-Umgebung auf Basis von Python 3.12 mit den benötigten Abhängigkeiten bereitstellt. Es dient der lokalen Reproduzierbarkeit der Notebook-Ausführung und ist nicht als produktionsreife Deployment-Umgebung zu verstehen. Der Container kann aus dem Repository-Wurzelverzeichnis mit `docker build -t cloudanobench-thesis .` gebaut und anschließend mit `docker run -p 8888:8888 cloudanobench-thesis` gestartet werden.